

Trabalho 03: Padrões de Projeto

O foco agora é entender as peças que compõem um sistema moderno e as regras para um código de alta qualidade.

Eixo 1: Componentes e APIs (Amanda)

Foco: Entender como as peças físicas e lógicas se conectam e como as APIs servem de ponte para o mundo.

1. Apresentação

- **O Trio:** Explicar o papel do **Front-end** (a interface/pele), **Back-end** (a inteligência/cérebro) e **Banco de Dados** (o armazenamento/memória). Mostrem como um depende do outro.
- **O Ecossistema de Rede:** Explicar para que servem as **CDNs** (entrega rápida de conteúdo) e os **Load Balancers** (equilíbrio de tráfego para o site não cair).
- **O Universo das APIs:** Detalhar o funcionamento de uma API e diferenciar os modelos:
 - **RESTful:** O padrão de "pergunta e resposta" mais comum.
 - **GraphQL:** Onde o cliente pede apenas o que precisa.
 - **WebSocket:** Para conversas em tempo real (ex: chat).
 - **Event-driven e SOAP:** O modelo baseado em eventos e o modelo clássico/robusto.

2. Atividade

- **APIs no Mundo Real:** O grupo deve trazer e demonstrar exemplos práticos de APIs que consumimos diariamente sem perceber (ex: API de localização no iFood, API de login com Google/Facebook ou API de música do Spotify).

3. Visão Prática (Livre Escolha)

- A equipe deve apresentar, da forma que preferir (ex: vídeo, encenação ou análise de caso), um exemplo de "Falha de Componente": O que acontece com um usuário quando a API cai, mas o Front-end continua no ar?

4. Texto Obrigatório para Leitura Dinâmica

Eixo 2: Princípios SOLID (Grazielle)

Foco: Aplicar as diretrizes que transformam um código confuso em um software profissional e fácil de manter.

1. Apresentação

- **Explorando o Acrônimo:** Explicar detalhadamente cada pilar do SOLID:
 - **S (Single Responsibility):** Uma classe, uma função.
 - **O (Open/Closed):** Evoluir sem precisar quebrar o que já existe.
 - **L (Liskov Substitution):** Subclasses que honram seus pais.
 - **I (Interface Segregation):** Não force ninguém a carregar o que não usa.
 - **D (Dependency Inversion):** Dependendo de abstrações, não de implementações fixas.

2. Atividade

- **SOLID no ScreenMatch:** Projetar o código do sistema **ScreenMatch** (desenvolvido em aula) e identificar onde os princípios foram aplicados. E como devem ser aplicados daqui pra frente (na perspectiva da equipe).

3. Visão Prática

- O grupo deve mostrar um exemplo de "Código Frágil" (um código que não segue o SOLID) e como uma pequena mudança nele causaria um desastre em outras partes do sistema. Apresentar um diagrama de classes comparativo: de um lado, uma "Classe Deus" (que faz tudo) e, do outro, a estrutura refatorada seguindo os princípios SOLID.

4. Texto Obrigatório para Leitura Dinâmica

Eixo 3: Design Patterns (Rian)

Foco: Conhecer as soluções testadas e aprovadas para problemas comuns de programação.

1. Apresentação

- **Categorias de Padrões:** Explicar a divisão entre padrões **Criacionais** (nascimento), **Estruturais** (montagem) e **Comportamentais** (atitude).
- **Os Padrões de Ouro:** Explicar o funcionamento e a utilidade de:
 - **Factory Method:** A fábrica de objetos.
 - **Singleton:** A conexão única.
 - **Adapter:** O tradutor de interfaces.
 - **Observer:** O sistema de alertas/notificações.
 - **Strategy:** A troca de inteligência conforme o contexto.

2. Atividade e Visão Prática

- Demonstrar em sala de aula através de uma analogia física como cada um os padrões funcionam. Ex: Usar um adaptador de tomada real para explicar o padrão **Adapter**, um problema matemático que pode ser resolvido de diversas maneiras para representar o padrão **Strategy**.

3. Texto Obrigatório para Leitura Dinâmica